LAB ASSIGNMENT 9.3

Documentation Generation – Automatic Documentation & Code Comments

Name:	**Neerudu Gifty Taniksha**	**Lab No:**	**Lab 9.3**
Roll No:	**2503A51107**	**Batch:**	**38**

LAB OBJECTIVES & OUTCOMES

Objectives: Understand docstrings and inline comments, leverage AI tools for doc generation, practice Google & NumPy formatting, and evaluate tools like pydoc.

TASK 1: BASIC DOCSTRING GENERATION & UTILITY FUNCTIONS

Writing structured function-level Google-style docstrings for a mathematical utility script and evaluating AI documentation quality.

> **AI PROMPT USED:**
> "Generate a Google-style Python docstring for a function that formats multiplication tables from 1 through 10."

Python Implementation with Google-Style Docstring

```
"""Print multiplication tables for the numbers 1 through 10.
Run this script to display all ten tables. No user input is required.
"""

def format_multiplication_table(number: int) -> list[str]:
    """Create the multiplication table for a number.

    Args:
        number: The number whose multiplication table will be created.

    Returns:
        A list of formatted table rows for multipliers 1 through 10.
    """
    return [
        f"{number} * {multiplier} = {number * multiplier}"
        for multiplier in range(1, 11)
    ]

def main() -> None:
    """Run the program and display all multiplication tables."""
    for number in range(1, 11):
        print(f"\nMultiplication table for {number}:")
        for row in format_multiplication_table(number):
            print(row)

if __name__ == "__main__":
    main()
```

Docstring Comparison Analysis

Aspect	Manual Docstring	AI-Generated Docstring
Formatting	Strictly adheres to Google Python Style Guide with `Args:` and `Returns:` headers.	Accurate Google-style formatting, often adds extra type annotations.
Clarity & Precision	Tailored specifically to domain logic and exact parameter ranges (1 to 10).	Generic description of data types without domain bounds.
Completeness	Focuses on key functionality and return structures cleanly.	High completeness, occasionally adds redundant exception tags.

TASK 1 EXECUTION SCREENSHOTS

```
tanik@LAPTOP-8SVR1F2C MINGW64 ~/OneDrive/Desktop/c_prog
$ C:/Users/tanik/.local/bin/python3.13.exe c:/Users/tanik/OneDrive/Desktop/c_prog/ai/mul.py

Multiplication table for 1:
1 * 1 = 1
1 * 2 = 2
1 * 3 = 3
1 * 4 = 4
1 * 5 = 5
1 * 6 = 6
1 * 7 = 7
1 * 8 = 8
1 * 9 = 9
1 * 10 = 10

Multiplication table for 2:
2 * 1 = 2
2 * 2 = 4
2 * 3 = 6
2 * 4 = 8
2 * 5 = 10
2 * 6 = 12
2 * 7 = 14
2 * 8 = 16
```

Figure 1.1: Output for Multiplication Tables 1 to 2

```
tanik@LAPTOP-8SVR1F2C MINGW64 ~/OneDrive/Desktop/c_prog
$ C:/Users/tanik/.local/bin/python3.13.exe c:/Users/tanik/OneDrive/Desktop/c_prog/ai/mul.py

Multiplication table for 3:
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12
3 * 5 = 15
3 * 6 = 18
3 * 7 = 21
3 * 8 = 24
3 * 9 = 27
3 * 10 = 30

Multiplication table for 4:
4 * 1 = 4
4 * 2 = 8
4 * 3 = 12
4 * 4 = 16
4 * 5 = 20
4 * 6 = 24
4 * 7 = 28
4 * 8 = 32
4 * 9 = 36
```

Figure 1.2: Output for Multiplication Tables 3 to 4

```
tanik@LAPTOP-8SVR1F2C MINGW64 ~/OneDrive/Desktop/c_prog
$ C:/Users/tanik/.local/bin/python3.13.exe c:/Users/tanik/OneDrive/Desktop/c_prog/ai/mul.py
5 * 10 = 50

Multiplication table for 6:
6 * 1 = 6
6 * 2 = 12
6 * 3 = 18
6 * 4 = 24
6 * 5 = 30
6 * 6 = 36
6 * 7 = 42
6 * 8 = 48
6 * 9 = 54
6 * 10 = 60

Multiplication table for 7:
7 * 1 = 7
7 * 2 = 14
7 * 3 = 21
7 * 4 = 28
7 * 5 = 35
7 * 6 = 42
7 * 7 = 49
7 * 8 = 56
```

Figure 1.3: Output for Multiplication Tables 6 to 7



Figure 1.4: Output for Multiplication Tables 9 to 10

TASK 2: AUTOMATIC INLINE COMMENTS & CODE LOGIC

Demonstrating inline comments explaining input validation, factorial calculation loops, and memory accumulation state.

> **AI PROMPT USED:**
> "Add clean inline comments explaining each logical block of this factorial calculation script."

Python Code with Manual Inline Comments

```
# Ask the user to enter a whole number.
number = int(input("Enter a non-negative integer: "))

# A factorial is not defined for negative integers.
if number < 0:
    print("Please enter a non-negative integer.")
else:
    # Start at 1 because multiplying by 1 does not change the result.
    factorial = 1

    # Multiply by every integer from 1 through the entered number.
    for value in range(1, number + 1):
        factorial *= value

    # Display the calculated factorial.
    print(f"{number}! = {factorial}")
```

Program Execution Output

```
tanik@LAPTOP-8SVR1F2C MINGW64 ~/OneDrive/Desktop/c_prog
$ C:/Users/tanik/.local/bin/python3.13.exe c:/Users/tanik/OneDrive/Desktop/c_prog/ai/fact.py
Enter a non-negative integer: 5
5! = 120
```

Figure 2.1: Execution Output for Factorial Calculation Program

Critical Discussion on AI-Generated Inline Comments

Strengths: AI tools excel at identifying mathematical operations and control loops instantly across complex codebases.

Limitations: AI often generates redundant comments (e.g., `x = x + 1 # increment x`) rather than explaining domain rationale.

TASK 3: MODULE-LEVEL & FUNCTION-LEVEL DOCUMENTATION

Command-line module documentation including error handling docstrings (`Raises`) and operator logic.

AI PROMPT USED:
"Generate a Google-style docstring for a Python calculator function handling basic arithmetic and division by zero."

Calculator Script Implementation

```
"""A simple command-line calculator for two numbers.

The calculator supports addition, subtraction, multiplication, and division.
Run this file and follow the prompts to choose an operation and enter numbers.
"""

def calculate(first_number: float, operator: str, second_number: float) -> float:
    """Perform a basic arithmetic operation on two numbers.

    Args:
        first_number: The left-hand number in the calculation.
        operator: One of ``+``, ``-``, ``*``, or ``/``.
        second_number: The right-hand number in the calculation.

    Returns:
        The result of the selected arithmetic operation.

    Raises:
        ValueError: If the operator is unsupported or division by zero is attempted.
    """
    if operator == "+":
        return first_number + second_number
    if operator == "-":
        return first_number - second_number
    if operator == "*":
        return first_number * second_number
    if operator == "/":
        if second_number == 0:
            raise ValueError("Cannot divide by zero.")
        return first_number / second_number
    raise ValueError("Unsupported operator. Choose +, -, *, or /.")


def main() -> None:
    """Prompt for a calculation and display its result."""
    first_number = float(input("Enter the first number: "))
    operator = input("Choose an operation (+, -, *, /): ").strip()
    second_number = float(input("Enter the second number: "))

    try:
        result = calculate(first_number, operator, second_number)
    except ValueError as error:
        print(f"Error: {error}")
    else:
        print(f"Result: {result}")

if __name__ == "__main__":
    main()
```

Calculator Execution Output

```
tanik@LAPTOP-8SVR1F2C MINGW64 ~/OneDrive/Desktop/c_prog
$ C:/Users/tanik/.local/bin/python3.13.exe c:/Users/tanik/OneDrive/Desktop/c_prog/ai/cal.py
Enter the first number: 2
Choose an operation (+, -, *, /): +
Enter the second number: 3
Result: 5.0
```

Figure 3.1: Execution Output for Calculator Program

ADVANCED TASK: AUTOMATED TERMINAL & HTML DOCUMENTATION WITH PYDOC

Python's built-in pydoc utility extracts docstrings directly from source code to generate terminal help pages or standalone HTML files.

> **AI PROMPT USED:**
> "Generate a Python module with pydoc-compatible docstrings and show how to view its documentation in terminal and export HTML."

Pydoc Multiplication Script (`mult_pydoc.py`)

```
"""Display multiplication tables for the numbers 1 through 5.

Run this module as a script to print five tables, each covering multipliers
from 1 through 10. Generate or view the documentation with Python's built-in ``pydoc`` module.
"""

def print_multiplication_tables() -> None:
    """Print multiplication tables for numbers 1 through 5.
    Each table displays the products of its number and the integers 1 through 10.
    """
    for number in range(1, 6):
        print(f"\nMultiplication table for {number}:")
        for multiplier in range(1, 11):
            print(f"{number} * {multiplier} = {number * multiplier}")

def main() -> None:
    """Run the program and print all five multiplication tables."""
    print_multiplication_tables()

if __name__ == "__main__":
    main()
```

Terminal Output (`python -m pydoc ai.mult_pydoc`)

```
tanik@LAPTOP-8SVR1F2C MINGW64 ~/OneDrive/Desktop/c_prog
$ python -m pydoc ai.mult_pydoc
Help on module ai.mult_pydoc in ai:

NAME
    ai.mult_pydoc - Display multiplication tables for the numbers 1 through 5.

DESCRIPTION
    Run this module as a script to print five tables, each covering multipliers
    from 1 through 10. Generate or view the documentation with Python's built-in
    ``pydoc`` module.

FUNCTIONS
    main() -> None
        Run the program and print all five multiplication tables.

        Returns:
            None.

    print_multiplication_tables() -> None
        Print multiplication tables for numbers 1 through 5.

        Each table displays the products of its number and the integers 1 through
```

Figure 4.1: Extracted Module Header and Description in Terminal

```
tanik@LAPTOP-8SVR1F2C MINGW64 ~/OneDrive/Desktop/c_prog
$ python -m pydoc ai.mult_pydoc
FUNCTIONS
    main() -> None
        Run the program and print all five multiplication tables.

        Returns:
            None.

    print_multiplication_tables() -> None
        Print multiplication tables for numbers 1 through 5.

        Each table displays the products of its number and the integers 1 through
        10.

        Returns:
            None.

FILE
    c:\users\tanik\onedrive\desktop\c_prog\ai\mult_pydoc.py
```

Figure 4.2: Function Signatures & File Metadata in Terminal Pydoc

HTML Export Command (`python -m pydoc -w ai.mult_pydoc`)

```
tanik@LAPTOP-8SVR1F2C MINGW64 ~/OneDrive/Desktop/c_prog
$ python -m pydoc -w ai.mult_pydoc
wrote ai.mult_pydoc.html
```

Figure 4.3: Terminal Command Writing `ai.mult_pydoc.html` File

Generated HTML Documentation Output

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Python: module ai.mult_pydoc</title>
</head><body>
<table class="heading">
<tr class="heading-text decor">
<td class="title"> <br><strong class="title"><a href="ai.html" class="white">ai</a>.mult_pydoc</s
trong></td>
<td class="extra"><a href=".">index</a><br><a href="file:///c:/users/tanik/onedrive/desktop/c_prog/ai/
mult_pydoc.py">mult_pydoc.py</a></td></tr></table>
<p><span class="code">Display multiplication tables for the numbers 1&nb
sp;through 5.</span></p>
<table class="section">
<tr class="decor functions-decor heading-text">
<td class="section-title" colspan=3> <br><strong class="bigsection">Functions</strong></td></tr>
<tr><td class="decor functions-decor"><span class="code">      </span></
td><td> </td>
<td class="singlecolumn"><dl><dt><a name="-main"><strong>main</strong></a>() -&gt; None</dt><dd><span
class="code">Run the program and print all five multiplication&nbs
p;tables.</span></dd></dl>
<dl><dt><a name="-print_multiplication_tables"><strong>print_multiplication_tables</strong></a>() -&g
t; None</dt><dd><span class="code">Print multiplication tables for numbers 1&
nbsp;through 5.</span></dd></dl>
</td></tr></table>
</body></html>
```

SUMMARY & CONCLUSION

By leveraging standardized Google docstrings and Python's built-in pydoc generator, developers can automate documentation workflows, improving code maintainability and output transparency across team environments.